Dream More Platform

Revised Software Requirements and Project Documentation

Technology Stack Revision — August 2026

Revision scope

The original requirements and product scope are retained. The technology, architecture, repository structure, deployment, security, testing, and development workflow sections have been revised to align with the selected modern stack.

1. Project Overview

Project Name

Dream More Platform

Project Type

Online Learning and Freelance Work Marketplace Platform

Vision

Dream More Platform is an integrated education and career development ecosystem that combines an online learning management system with a freelance and job marketplace. The platform enables students to learn skills through courses, instructors to create and sell courses, and graduates/freelancers to find work opportunities based on acquired skills and verified experience.

Business Objectives

- Provide quality online education.
- Allow qualified teachers to create and sell courses.
- Enable administrators to monitor platform activities.
- Provide secure payment verification.
- Help students transition into employment.
- Create a marketplace where graduates can offer services.
- Connect employers with skilled graduates.
- Track learning progress and career development.

2. User Roles

Administrator

- Manage users, teachers, students, employers, and freelancers.
- Approve or reject teacher applications.
- Approve or reject courses.
- Verify payment submissions.
- Verify teachers, freelancers, and employers.
- Monitor jobs and resolve disputes.
- View revenue, student, teacher, course, and marketplace reports.

Student

- Register, log in, manage a profile, verify email, and recover a password.
- Browse, search, filter, enroll in, and review courses.
- Watch lessons, download resources, complete quizzes, and track progress.
- Submit payment screenshots and transaction IDs.
- Receive certificates after completing required learning and assessments.
- Receive platform notifications.

Teacher / Instructor

- Apply with personal information, CV/resume, credentials, certificates, experience documents, and portfolio.

- Wait for administrator approval.
- Create and manage courses, sections, lessons, quizzes, resources, thumbnails, and videos.
- View students, revenue, reviews, and course analytics.
- Submit courses for administrator approval.

Employer

- Create an account and verify identity.
- Post jobs with title, description, budget, and deadline.
- Review proposals and hire freelancers.
- Review submitted work, make payments, and leave ratings/reviews.

Freelancer / Graduate

- Create a professional profile and portfolio.
- Display skills, certifications, experience, ratings, and reviews.
- Apply for jobs with proposals, estimated cost, and portfolio information.
- Execute work, submit deliverables, and receive ratings/reviews.

3. Platform Modules

3.1 Learning Management System (LMS)

- Course categories may include Programming, Graphic Design, Video Editing, Digital Marketing, Mechanical Engineering, Electrical Engineering, Civil Engineering, Business, and Language Learning.
- Course structure: Course → Sections → Lessons → Videos/Documents → Assignments → Quizzes.
- Track completed lessons, completion percentage, quiz scores, and certificates earned.
- Students can browse, search, filter, watch lessons, download resources, complete quizzes, receive certificates, and review courses.

3.2 Payment Verification System

- Student selects a course.
- System displays payment instructions.
- Student uploads a payment screenshot and transaction ID.
- Administrator reviews the submission.
- Administrator approves or rejects the payment.
- Student receives a notification.
- Payment states: Pending, Approved, Rejected.

3.3 AppWorks Freelance Marketplace

- Freelancer profiles include personal information, location, skills, experience, certifications, portfolio, completed projects, ratings, reviews, and success rate.
- Job categories include Web Development, Mobile Development, UI/UX Design, Video Editing, Graphic Design, Animation, Mechanical Design, CAD Modeling, Simulation, Marketing, Data Entry, and Virtual Assistant.
- Job workflow: Employer posts job → Freelancer applies → Employer reviews → Work execution → Freelancer submits work → Employer reviews → Completion → Rating and review.

3.4 Verification System

- Teacher verification: degree, certificates, CV, and experience letters.

- Freelancer verification: national ID, certificates, and portfolio.
- Employer verification: company registration, national ID, and contact information.

3.5 Certificate System

- Certificates are generated automatically after required completion and assessment conditions are met.
- Certificate fields: Student Name, Course Name, Instructor Name, Completion Date, Unique Certificate ID, and QR Verification Code.

3.6 Notification System

- Student notifications: payment approved, course approved, certificate earned, and new course released.
- Teacher notifications: application approved, course approved, and new enrollment.
- Employer notifications: new proposal and work submitted.
- Freelancer notifications: job invitation and proposal accepted.
- Real-time in-app notifications are supported through WebSockets/Socket.IO.

4. Dashboard Requirements

Student Dashboard

- Continue Learning
- Trending Courses
- Recommended Courses
- Latest Courses
- Certificates
- Learning Statistics

Teacher Dashboard

- Total Students
- Revenue
- Courses
- Reviews
- Pending Courses

Admin Dashboard

- Total Users
- Total Students
- Total Teachers
- Total Courses
- Total Revenue
- Pending Payments
- Pending Course Approvals
- Pending Teacher Approvals
- Active Jobs

Employer Dashboard

- Posted Jobs

- Active Jobs
- Completed Jobs
- Payments

Freelancer Dashboard

- Applied Jobs
- Active Jobs
- Earnings
- Ratings

5. Database Design

Core Entities

- User
- StudentProfile
- TeacherProfile
- EmployerProfile
- FreelancerProfile
- Course
- CourseSection
- Lesson
- LessonResource
- Quiz
- QuizQuestion
- QuizAttempt
- Enrollment
- Payment
- Certificate
- Job
- Proposal
- Project / WorkSubmission
- Review
- VerificationRequest
- Notification

Relationships

- User has one role and may have one role-specific profile.
- Teacher creates many Courses.
- Course contains many Sections; each Section contains many Lessons.
- Student has many Enrollments; each Enrollment belongs to one Course.
- Payment belongs to a Student and Course and is linked to an Enrollment after approval.
- Certificate belongs to a Student and Course and has a unique verification code.

- Employer creates many Jobs.
- Job receives many Proposals from Freelancers.
- Jobs may progress through posted, active, submitted, completed, rejected, or disputed states.
- Reviews are associated with completed courses or completed marketplace work.

6. Technology Stack

Frontend

- Next.js 16 with TypeScript.
- Tailwind CSS for styling.
- shadcn/ui for reusable accessible UI components.
- TanStack Query for server-state/data fetching and caching.
- Zustand for lightweight client-side application state.
- React Hook Form for form handling.
- Zod for schema validation.
- Lucide React for icons.
- Recharts for dashboards and analytics.
- Next.js built-in routing and rendering capabilities.
- Vercel for frontend deployment.

Backend

- NestJS with TypeScript.
- REST API for application services.
- WebSockets / Socket.IO for real-time notifications and events.
- JWT access tokens with refresh-token based session renewal.
- NestJS guards, pipes, interceptors, and modules for authentication, authorization, validation, logging, and cross-cutting concerns.
- Jest for backend unit/integration testing.

Database

- PostgreSQL as the primary relational database.
- Prisma ORM for type-safe database access, migrations, and schema management.

Storage and Media

- Cloudinary or S3-compatible object storage for images, documents, certificates, and uploaded files.
- Cloudinary or Mux for course video delivery and media processing, depending on final media requirements.
- Private/signed URLs should be used for protected learning resources where appropriate.

Communication

- Resend or another transactional email provider for verification, password reset, and platform emails.
- WebSockets / Socket.IO for real-time in-app notifications.

Certificates

- PDF certificate generation service/library.
- QR code generation for certificate verification.

- Public verification endpoint/page that validates a certificate by its unique ID/code.

Development and Deployment

- Docker for reproducible local development and deployment.
- Git and GitHub for version control and collaboration.
- Vercel for the Next.js web application.
- Railway or Render for the NestJS API and PostgreSQL environments.
- Separate development/staging and production environments.
- Environment variables for secrets and environment-specific configuration.

7. System Architecture

Architecture

- The platform uses a layered architecture separating the web application, backend API, database, and object/media storage.
- Next.js provides the public web experience, authenticated dashboards, server rendering where useful, and frontend UI.
- NestJS provides the application API and central business logic.
- PostgreSQL stores transactional and relational platform data.
- Prisma provides type-safe database access and migrations.
- Cloudinary/S3-compatible storage handles uploaded assets and protected files.
- WebSockets/Socket.IO provides real-time notification delivery.

Request Flow

- Browser → Next.js → NestJS REST API → Prisma → PostgreSQL.
- Next.js and NestJS communicate through authenticated API requests.
- Media upload/download flows use the configured storage provider rather than storing large media directly in PostgreSQL.
- Real-time events flow from NestJS to connected clients through WebSockets/Socket.IO.

8. Repository and Project Structure

Recommended Repository

- Use a monorepo for the platform to keep shared types, validation schemas, and development tooling consistent.
- apps/web — Next.js frontend.
- apps/api — NestJS backend.
- packages/shared — shared TypeScript types and validation contracts where appropriate.
- packages/config — shared linting/TypeScript/configuration if required.
- docs — project documentation, API documentation, database design, sprint reports, and meeting notes.

Frontend Structure

- app/ — Next.js routes and layouts.
- components/ — reusable UI components.
- features/ — domain-oriented feature modules.
- lib/ — API client, utilities, authentication helpers, and configuration.

- hooks/ — reusable React hooks.
- schemas/ — Zod schemas.
- stores/ — Zustand stores.
- types/ — frontend types.

Backend Structure

- src/modules/auth
- src/modules/users
- src/modules/courses
- src/modules/enrollments
- src/modules/payments
- src/modules/certificates
- src/modules/jobs
- src/modules/proposals
- src/modules/reviews
- src/modules/verifications
- src/modules/notifications
- src/common
- src/config
- prisma/

9. Security, Testing, and Development Workflow

Security

- Use strong password hashing and never store plaintext passwords.
- Use JWT access tokens with refresh-token rotation/expiry policies.
- Enforce role-based authorization at the NestJS API layer.
- Validate all request payloads using DTOs and validation pipes; use Zod where shared client/server validation is beneficial.
- Apply rate limiting to authentication and sensitive endpoints.
- Use secure HTTP headers, CORS policy, CSRF protection where applicable to the chosen authentication flow, and strict input validation.
- Protect uploaded files and restrict file types and sizes.
- Use signed/private URLs for protected media and documents where required.
- Keep secrets only in environment variables or managed secret storage.

Testing

- Jest for backend unit and integration tests.
- Playwright for end-to-end tests covering critical user journeys.
- Test authentication, role permissions, course enrollment, payment approval, certificate generation, job proposals, and notification flows.
- Run linting, type checking, tests, and build checks before merging.

Development Workflow

- Use feature branches such as feature/authentication, feature/student-dashboard, feature/teacher-module, feature/payment-module, feature/admin-module, feature/appworks-module.
- Create pull requests for review.
- Merge reviewed work into the development branch.
- Run automated checks before merging.
- Deploy development/staging builds for integration and QA testing.
- Promote tested releases to production.

10. Development Phases and MVP

Development Phases

- Phase 1 (Weeks 1–4): Project setup, database design, authentication, user roles, authorization, and dashboard layouts.
- Phase 2 (Weeks 5–8): Teacher verification, course creation, course management, and admin approval.
- Phase 3 (Weeks 9–12): Payment screenshot workflow, enrollment, video learning, and learning progress.
- Phase 4 (Weeks 13–16): Certificate generation, QR verification, reviews, and ratings.
- Phase 5 (Weeks 17–20): AppWorks marketplace, job posting, proposals, hiring, work submission, and completion.
- Phase 6 (Weeks 21–24): Analytics, security hardening, testing, performance optimization, deployment, and production readiness.

MVP Scope

- Student registration and login.
- Teacher application and approval.
- Course creation and admin approval.
- Course enrollment.
- Screenshot-based payment verification.
- Video learning.
- Learning progress tracking.
- Certificates with unique verification codes and QR verification.
- Student dashboard.
- Admin dashboard.
- Basic AppWorks job marketplace.
- Basic notifications.

Production Readiness

- Configure production environment variables and secrets.
- Configure backups and database recovery procedures.
- Configure object storage and protected media delivery.
- Set up monitoring, structured logging, error tracking, and audit logs.
- Set up CI/CD for tests, builds, and controlled deployment.
- Document API endpoints, database migrations, environment setup, and operational procedures.

Technology Stack Summary

Layer	Selected Technology
Frontend	Next.js 16 + TypeScript
UI	Tailwind CSS + shadcn/ui
Backend	NestJS + TypeScript
API	REST + WebSockets/Socket.IO
Database	PostgreSQL
ORM	Prisma
Authentication	JWT + Refresh Tokens
Validation	NestJS validation + Zod
Server State	TanStack Query
Client State	Zustand
Forms	React Hook Form
Media	Cloudinary / Mux
Object Storage	Cloudinary / S3-compatible storage
Email	Resend
Certificates	PDF generation + QR verification
Testing	Jest + Playwright
Containers	Docker
Deployment	Vercel + Railway/Render
Version Control	Git + GitHub

Architecture decision: Next.js is responsible for the web experience and rendering; NestJS is the dedicated business/API layer. This keeps the backend independently consumable by future clients while preserving a unified TypeScript ecosystem.